

Deadlock in C# with Example

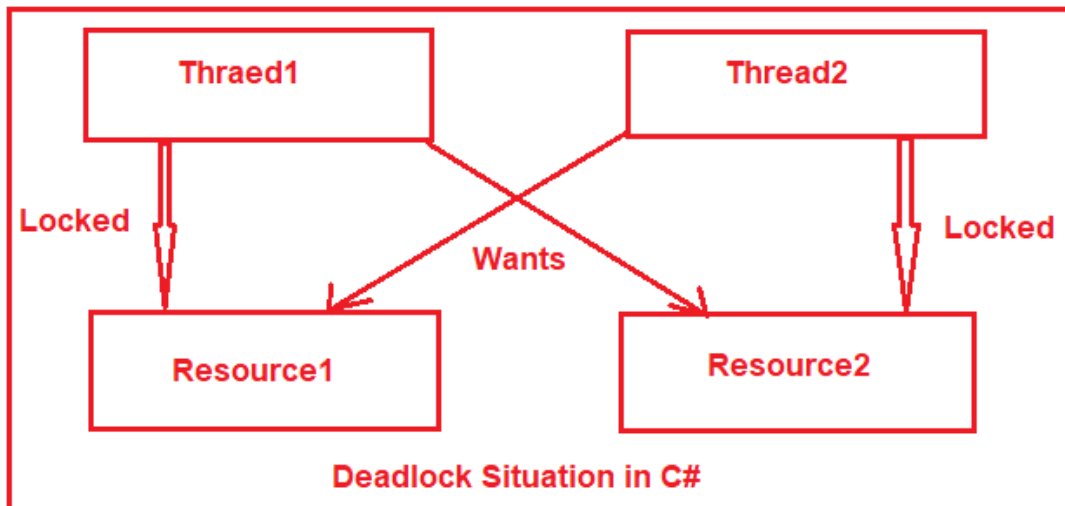
In this article, I am going to discuss **Deadlock in C#** with Examples. Please read our previous article discussing [SemaphoreSlim in C#](#) with Examples. Deadlock is one of the most important aspects to understand as a developer. As part of this article, we are going to discuss the following pointers.

1. **What is deadlock?**
2. **Why did a Deadlock occur?**
3. **How a deadlock can occur in a multithreaded application?**
4. **How to avoid a Deadlock by using Monitor.TryEnter method?**
5. **How Do We Avoid Deadlock by acquiring locks in a specific order?**

What is a Deadlock in C#?

In simple words, we can define a deadlock in C# as a situation where two or more threads are **unmoving or frozen** in their execution because they are waiting for each other to finish.

For example, let's say we have two threads, Thread1 and Thread2, and two resources, Resource1 and Resource2. Thread1 locked **Resource1** and tried to acquire a lock on **Resource2**. At the same time, **Thread2** acquired a lock on **Resource2** and tried to acquire a lock on **Resource1**.



As you can see in the above image, **Thread1** is waiting to acquire a **lock on Resource2**, which is held by **Thread2**. **Thread2** also can't finish its work and release the lock on **Resource2** because it is waiting to acquire a lock on **Resource1**, which Thread1 locks, and hence a Deadlock situation occurred.

Deadlock can occur if the following conditions hold true:

1. **Mutual Exclusion:** This implies that only one thread can access a resource at a particular time.
2. **Hold and Wait:** This is a condition in which a thread holds at least one resource and waits for at least one resource already acquired by another thread.
3. **Circular Wait:** This is a condition in which two or more threads are waiting for a resource acquired by the next member in the chain.

Example to understand Deadlock in C#:

Let's understand Deadlock in C# with an example. Create a class file named **Account.cs** and copy and paste the following code into it.

```
namespace DeadLockDemo
{
    public class Account
    {
        public int ID { get; }
        private double Balance { get; set;}
        public Account(int id, double balance)
        {
            ID = id;
            Balance = balance;
        }

        public void WithdrawMoney(double amount)
        {
            Balance -= amount;
        }
        public void DepositMoney(double amount)
        {
            Balance += amount;
        }
    }
}
```

The above Account class is very straightforward. We created the class with two properties, ID and Balance, which we initialize through the constructor of this class. So, at the time of creating the Account class instance, we need to pass the ID and Balance value. We have also created two methods. The WithdrawMoney method is used to withdraw the amount, while the DepositMoney method is used to add the amount.

AccountManager.cs:

Next, create another class file named **AccountManager.cs** and copy and paste the following code into it.

```
using System;
using System.Threading;
namespace DeadLockDemo
{
    public class AccountManager
    {
        private Account FromAccount;
        private Account ToAccount;
        private double TransferAmount;
        public AccountManager(Account AccountFrom, Account AccountTo, double
AmountTransfer)
        {
            FromAccount = AccountFrom;
```

```

        ToAccount = AccountTo;
        TransferAmount = AmountTransfer;
    }
    public void FundTransfer()
    {
        Console.WriteLine($"{Thread.CurrentThread.Name} trying to acquire
{FromAccount.ID}");
        lock (FromAccount)
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} acquired lock
{FromAccount.ID}");
            Console.WriteLine($"{Thread.CurrentThread.Name} Doing Some wor
Thread.Sleep(1000);
            Console.WriteLine($"{Thread.CurrentThread.Name} trying to acqu
{ToAccount.ID}");
            lock (ToAccount)
            {
                FromAccount.WithdrawMoney(TransferAmount);
                ToAccount.DepositMoney(TransferAmount);
            }
        }
    }
}

```

In the above code, we created two Account type variables to hold the FromAccount and ToAccount details, i.e., the Account from where the amount is going to be deducted and the account to whom the amount is created. We also created another double-type variable, i.e., TransferAmount, to hold the amount that is going to be deducted from the FromAccount and credited to the ToAccount. Through the constructor of this class, we are initializing these variables.

We also created the FundTransfer method, which will perform the required task. As you can see, it first acquires a lock on the From Account and then does some work. After 1 second, it backs up and tries to acquire a lock on the To Account.

Modifying the Main Method:

Now modify the Main method of the Program class as shown below. Here, for accountManager1, Account1001 is the FromAccount, and Account1002 is the ToAccount. Similarly, for accountManager2, Account1002 is the FromAccount and Account1001 is the ToAccount

```

using System;
using System.Threading;
namespace DeadLockDemo
{
    class Program
    {
        public static void Main()
        {
            Console.WriteLine("Main Thread Started");
        }
    }
}

```

```

Account Account1001 = new Account(1001, 5000);
Account Account1002 = new Account(1002, 3000);
AccountManager accountManager1 = new AccountManager(Account1001, /
5000);

Thread thread1 = new Thread(accountManager1.FundTransfer)
{
    Name = "Thread1"
};
AccountManager accountManager2 = new AccountManager(Account1002, /
6000);

Thread thread2 = new Thread(accountManager2.FundTransfer)
{
    Name = "Thread2"
};
thread1.Start();
thread2.Start();
thread1.Join();
thread2.Join();
Console.WriteLine("Main Thread Completed");
Console.ReadKey();
    }
}
}

```

Output:

```

Main Thread Started
Thread1 trying to acquire lock on 1001
Thread1 acquired lock on 1001
Thread1 Doing Some work
Thread2 trying to acquire lock on 1002
Thread2 acquired lock on 1002
Thread2 Doing Some work
Thread1 trying to acquire lock on 1002
Thread2 trying to acquire lock on 1001

```

Note: For thread1, Account1001 is resource1 and Account1002 is resource2. On the other hand, for thread2, Account1002 is resource1, and Account1001 is resource2. Now, run the application and see if a deadlock occurred.

The reason for the deadlock is that **thread1** acquired an exclusive lock on **Account1001** and then did some processing. In the meantime, **thread2** started, and it acquired an exclusive lock on **Account1002** and then did some processing. Then **thread1** back and wanted to acquire a lock on **Account1002** which is already locked by thread2. Similarly, **thread2** is back and wants to acquire a lock on Account1001, which is already locked by thread1 and hence deadlock.

Avoiding Deadlock by using Monitor.TryEnter method?

One of the overloaded versions (**TryEnter(object obj, int millisecondsTimeout)**) of the **Monitor.TryEnter** method takes the second parameter as the time out in milliseconds. Using that parameter, we can specify a timeout for the thread to release the lock. If a thread is holding a resource for a long time while the other thread is waiting, then Monitor will provide a time limit and force the lock to release it. So that the other thread can enter the critical section. Modifying the **AccountManager** class as shown below:

```
using System;
using System.Threading;
namespace DeadLockDemo
{
    public class AccountManager
    {
        private Account FromAccount;
        private Account ToAccount;
        private double TransferAmount;
        public AccountManager(Account AccountFrom, Account AccountTo, double
AmountTransfer)
        {
            this.FromAccount = AccountFrom;
            this.ToAccount = AccountTo;
            this.TransferAmount = AmountTransfer;
        }
        public void FundTransfer()
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} trying to acquire
{FromAccount.ID}");

            lock (FromAccount)
            {
                Console.WriteLine($"{Thread.CurrentThread.Name} acquired lock
{FromAccount.ID}");
                Console.WriteLine($"{Thread.CurrentThread.Name} Doing Some wor
Thread.Sleep(3000);
                Console.WriteLine($"{Thread.CurrentThread.Name} trying to acqui
{ToAccount.ID}");

                if (Monitor.TryEnter(ToAccount, 3000))
                {
                    Console.WriteLine($"{Thread.CurrentThread.Name} acquired l
{ToAccount.ID}");

                    try
                    {
                        FromAccount.WithdrawMoney(TransferAmount);
                        ToAccount.DepositMoney(TransferAmount);
                    }
                    finally
                    {
                        Monitor.Exit(ToAccount);
                    }
                }
            }
        }
    }
}
```

```

        else
        {
            Console.WriteLine($"{Thread.CurrentThread.Name} Unable to
lock on {ToAccount.ID}, So existing.");
        }
    }
}
}
}
}

```

Output:

```

Main Thread Started
Thread1 trying to acquire lock on 1001
Thread1 acquired lock on 1001
Thread1 Doing Some work
Thread2 trying to acquire lock on 1002
Thread2 acquired lock on 1002
Thread2 Doing Some work
Thread1 trying to acquire lock on 1002
Thread2 trying to acquire lock on 1001
Thread1 Unable to acquire lock on 1002, So existing.
Thread2 acquired lock on 1001
Main Thread Completed

```

As you can see in the output, thread1 releases the lock and exits from the critical section, which allows thread2 to enter the critical section and complete its task.

How Do We Avoid Deadlock in C# by acquiring locks in a specific order?

Please modify the AccountManager class as shown below.

```

using System;
using System.Threading;
namespace DeadLockDemo
{
    public class AccountManager
    {
        private Account FromAccount;
        private Account ToAccount;
        private readonly double TransferAmount;
        private static readonly Mutex mutex = new Mutex();
        public AccountManager(Account AccountFrom, Account AccountTo, double
AmountTransfer)
        {
            this.FromAccount = AccountFrom;
            this.ToAccount = AccountTo;
            this.TransferAmount = AmountTransfer;
        }
    }
}

```


In the next article, I am going to show you the [Performance of a multithreaded program when running on a single-core/processor machine versus a multi-core/processor](#) machine. In this article, I try to explain deadlock in C# using different approaches. I hope you enjoy this Deadlock in C# with Examples article.